In [64]:
```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import accuracy_score, mean_squared_error
```

In [70]:
```
data = pd.read_csv('pima-indians-diabetes.csv')
df = pd.DataFrame(data=data.values, columns=['Pregnancies', 'Glucose', 'Blood_Pr
df.head(10)
```

Out[70]:

	Pregnancies	Glucose	Blood_Pressure	Skin_Thickness	Insulin	Bmi	Diabetes_Pedigre
0	1.0	85.0	66.0	29.0	0.0	26.6	0.35
1	8.0	183.0	64.0	0.0	0.0	23.3	0.67
2	1.0	89.0	66.0	23.0	94.0	28.1	0.16
3	0.0	137.0	40.0	35.0	168.0	43.1	2.28
4	5.0	116.0	74.0	0.0	0.0	25.6	0.20
5	3.0	78.0	50.0	32.0	88.0	31.0	0.24
6	10.0	115.0	0.0	0.0	0.0	35.3	0.13
7	2.0	197.0	70.0	45.0	543.0	30.5	0.15
8	8.0	125.0	96.0	0.0	0.0	0.0	0.23
9	4.0	110.0	92.0	0.0	0.0	37.6	0.19

In [ ]:
```
x = [8.0, 183.0, 64.0, 0.0, 0.0, 23.3, 0.672, 32.0]
```

In [3]:
```
ohe = LabelEncoder()
Y = df['Prediction'].astype(int)
```

In [4]:
```
X = df.drop(columns='Prediction')
sc = StandardScaler()
X = sc.fit_transform(X)
```

In [5]:
```
x_train, x_test, y_train, y_test = train_test_split(X, Y, test_size=0.3)
```

In [6]:
```
x_test
```

Out[6]:
```
array([[ 0.64053533,  2.16351589,  1.18303717, ...,  0.44514732,
        -0.58462135,  2.79114451],
       [ 0.343683  , -1.49758333, -0.47023757, ..., -0.65836094,
        -0.61480719, -0.52952151],
       [ 1.5310923 ,  1.03702382,  0.45972947, ..., -0.13831682,
        -0.9287399 ,  1.00309358],
       ...,
       [ 0.64053533,  0.94314949, -0.3669079 , ...,  0.44514732,
         0.66507232, -0.44437623],
       [-0.84372629,  1.88189288, -0.26357823, ...,  0.26757128,
        -0.43369217,  0.4070766 ],
       [-0.54687397, -1.18466886, -0.21191339, ...,  0.61003936,
         0.47490154, -0.78495736]], shape=(231, 8))
```

In [59]:
```
clf = RandomForestClassifier(n_estimators=100,
                             criterion='gini',
                             max_depth=10,
                             max_features='log2',
                             min_samples_split=7,
                             min_samples_leaf=2,
                             bootstrap=True,
                             n_jobs=-1)
model = clf.fit(x_train, y_train)
pred = model.predict(x_test)
```

In [27]:
```
y_test.index = np.arange(len(y_test.index))
y_test.index
```

Out[27]:
```
Index([  0,   1,   2,   3,   4,   5,   6,   7,   8,   9,
       ...
       221, 222, 223, 224, 225, 226, 227, 228, 229, 230],
      dtype='int64', length=231)
```

In [60]:
```
sklearn_accuracy = accuracy_score(y_test, pred)
print(f"l'accurece score est {sklearn_accuracy * 100}")
```

```
l'accurece score est 77.48917748917748
```

In [67]:
```
for i in range(0, len(y_test), 10):
    print(f"{y_test[i]}   and  {pred[i]}")
```

```
1   and  1
0   and  1
1   and  1
0   and  0
0   and  0
1   and  0
0   and  0
0   and  0
1   and  0
0   and  0
0   and  0
0   and  0
0   and  1
1   and  1
0   and  0
0   and  0
0   and  0
0   and  0
1   and  1
0   and  0
0   and  0
0   and  0
1   and  0
0   and  0
```

In [76]:
```
np.array(df.loc[2][:-1])
```

Out[76]:
```
array([ 1.   , 89.   , 66.   , 23.   , 94.   , 28.1  ,  0.167, 21.   ])
```

In [77]:
```
# test le model avec une nouvelle valeur
predections = []
for i in range(10) :
    x = np.array(df.loc[i][:-1])
    pr = model.predict(sc.transform([x]))
    predections.append(pr)

for i in range(10):
    print(f"pour l'index {i} la prediction est {predections[i]} et la valeur ree
```

```
d:\ai_ml_env\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X d
oes not have valid feature names, but StandardScaler was fitted with feature name
s
  warnings.warn(
d:\ai_ml_env\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X d
oes not have valid feature names, but StandardScaler was fitted with feature name
s
  warnings.warn(
```

```
d:\ai_ml_env\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X d
oes not have valid feature names, but StandardScaler was fitted with feature name
s
  warnings.warn(
d:\ai_ml_env\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X d
oes not have valid feature names, but StandardScaler was fitted with feature name
s
  warnings.warn(
d:\ai_ml_env\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X d
oes not have valid feature names, but StandardScaler was fitted with feature name
s
  warnings.warn(
d:\ai_ml_env\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X d
oes not have valid feature names, but StandardScaler was fitted with feature name
s
  warnings.warn(
d:\ai_ml_env\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X d
oes not have valid feature names, but StandardScaler was fitted with feature name
s
  warnings.warn(
d:\ai_ml_env\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X d
oes not have valid feature names, but StandardScaler was fitted with feature name
s
  warnings.warn(
pour l'index 0 la prediction est [0] et la valeur reel est 0.0
pour l'index 1 la prediction est [1] et la valeur reel est 1.0
pour l'index 2 la prediction est [0] et la valeur reel est 0.0
pour l'index 3 la prediction est [1] et la valeur reel est 1.0
pour l'index 4 la prediction est [0] et la valeur reel est 0.0
pour l'index 5 la prediction est [0] et la valeur reel est 1.0
pour l'index 6 la prediction est [0] et la valeur reel est 0.0
pour l'index 7 la prediction est [1] et la valeur reel est 1.0
pour l'index 8 la prediction est [0] et la valeur reel est 1.0
pour l'index 9 la prediction est [0] et la valeur reel est 0.0
d:\ai_ml_env\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X d
oes not have valid feature names, but StandardScaler was fitted with feature name
s
  warnings.warn(
d:\ai_ml_env\Lib\site-packages\sklearn\utils\validation.py:2749: UserWarning: X d
oes not have valid feature names, but StandardScaler was fitted with feature name
s
  warnings.warn(
```

In [65]:
```
mse = mean_squared_error(y_test, pred)
mse
```

Out[65]:
```
0.22510822510822512
```

In [ ]:
```
import pickle

with open('randomForest_indians_diabete.pkl', 'wb') as f:
    pickle.dump(model, f)
```

changement de nombre d'observations

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]:

In [ ]: